SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS

Fall Semester 2026

Course: BCSE306L - Artificial Intelligence
Activity: Course Project Review - 2 | **Class ID:** VL20262705001
Date of Review: 13 / 10 / 2026

AUTOMATIC EVALUATION OF HANDWRITTEN ANSWER SHEETS

Presented by Student Team:

S. No.	Student Name	Register Number	Project Role & Specialization
1	**D. Sathvik**	24BDE0174	Lead System Architect & Deep Learning Engineer
2	**K. Devendra**	24BDE0148	Full-Stack Web UI/UX & Verification Engineer
3	**R. V. Guru Charan**	24BDE0075	Computer Vision & Slicing Pipeline Engineer

Under the Esteemed Guidance of

Prof. Nivedita S

Assistant Professor, SCORE

PROJECT SNAPSHOT

PART A – Project Overview

1. Project Particulars

Particular	Details
Project Title	Automatic Evaluation of Handwritten Answer Sheets
Area of Application	Educational technology: automated examination assessment
Scope	Automatic grading of handwritten True / False / Empty answer sheets with 10 questions per sheet: orientation correction, answer-column and row detection, Vision Transformer classification, scoring, class ranking and CSV export. The current scope covers three answer types (True, False, Empty).

2. Alignment with Sustainable Development Goals (SDGs)

SDG No.	SDG Title	Justification
SDG 4	Quality Education	Speeds up and standardises assessment, giving students faster and more consistent feedback and freeing teachers' time for teaching.
SDG 9	Industry, Innovation and Infrastructure	Applies deep learning and computer vision to modernise examination infrastructure in educational institutions.

3. Inputs – Features – Outputs

S. No.	Feature	Input Given	Output
1	Orientation and preprocessing	Raw sheet photo or scan from any camera or scanner	Upright image, longest side 900 px
2	Answer-column detection and row slicing	Upright sheet image	10 cell crops (C1 to C10)
3	Answer classification (ViT-B/16)	Each cell crop resized to 224 x 224 x 3	Class Empty / False / True with confidence
4	Scoring, ranking and export	Predicted answers, answer key, class roster	Score, percentage, grade, rank, ranked CSV
Final Output		Graded result (score, percentage, grade) for every student sheet and a ranked class leaderboard exported as CSV.	

4. Demo Video

Particular	Details
Video URL	https://drive.google.com/file/d/12gc56Sjc0x0mXaKbEBxthbK8EB3xyGHW/view?usp=drivesdk
Duration	02:49

5. Technical Stack

Layer	Technology / Tool
Frontend	HTML / CSS / JavaScript single-page app (dark and light glass UI)
Backend	Python, FastAPI (/api/evaluate, evaluate-students, CSV export)

Database	None; results are exported as ranked CSV
AI / ML Models & Libraries	PyTorch ViT-B/16 (343 MB weights), Keras CNN baseline, OpenCV (resize, rotate, Canny, Hough)
Tools & IDE	Git and Git LFS; launchers run_app.bat and run_server.py
Deployment / Hosting	Runs locally through the launchers (CPU inference)

6. Architecture Diagram

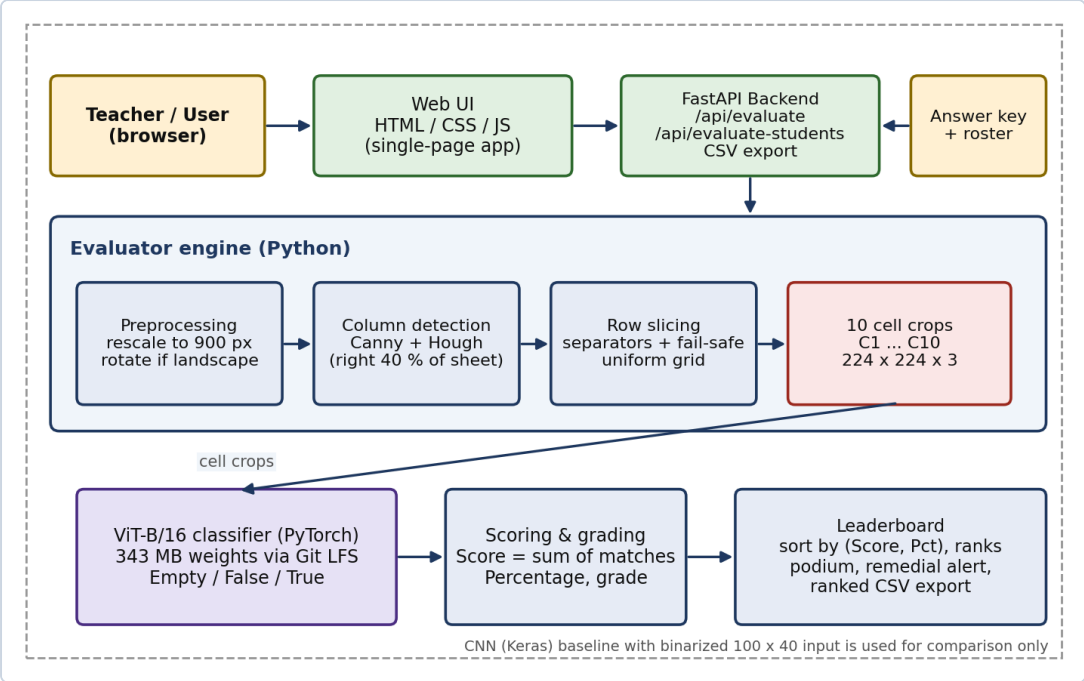


Fig. 1: System architecture of the handwritten answer-sheet evaluator

PART B – Team and Project Journey

1. Division of Work and Skills

S.No.	Student Name	Reg. No.	Work Done	Technical Skills Already Known	Skills, Tools & Knowledge Acquired
1	**D. Sathvik**	24BDE0174	Lead system architect and deep learning engineer: designed the end-to-end pipeline, built and integrated the ViT-B/16 classifier and the CNN baseline, and defined the scoring and ranking flow.	Python, Deep Learning fundamentals, PyTorch basics, Git	PyTorch, Vision Transformers (ViT-B/16), Keras CNN, system design
2	**K. Devendra**	24BDE0148	Full-stack web UI/UX and verification engineer: built the single-page web interface (dark and light glass UI), connected it to the FastAPI endpoints and verified results end to end.	HTML, CSS, JavaScript, Web development basics, REST APIs	HTML / CSS / JavaScript, FastAPI integration, testing and verification
3	**R. V. Guru Charan**	24BDE0075	Computer vision and slicing: implemented rescaling and orientation correction, Canny and Hough answer-column detection, row-separator detection and the fail-safe grid that always yields 10 cells.	Python, OpenCV basics, Image Processing fundamentals, Linear Algebra	OpenCV, Canny edge detection, Hough transform, image slicing

2. Phases and Plan

Phase	Activities	Timeline (From – To)	Status
Phase 1: Design and preprocessing	Problem study, sheet format analysis, rescaling and orientation correction	15/08/2026 – 31/08/2026	Completed
Phase 2: Cell extraction	Hough column detection, row-separator detection, fail-safe uniform grid	01/09/2026 – 25/09/2026	Completed
Phase 3: Classification models	ViT-B/16 classifier and CNN baseline for Empty / False / True	26/09/2026 – 20/10/2026	Completed
Phase 4: Application and testing	FastAPI backend, web UI, scoring, leaderboard, CSV export and verification	21/10/2026 – 02/11/2026	Completed

3. Difficulties Faced in Implementation

S.No.	Difficulty Faced	Where It Went Wrong	How It Was Corrected
1	Line detection broke on creased or poorly lit sheets	Row separators were missed or merged, so fewer than 10 cells were found	Added a fail-safe uniform grid (y_q = y_start + q(y_end - y_start)/10) so exactly 10 cells are always produced
2	Photos taken in landscape orientation	Column and row detection assumed an upright sheet	Rescale to 900 px, then rotate by 90 degrees whenever height is less than width
3	Large ViT weights file (343 MB)	Could not be stored in a normal Git repository	Stored the weights with Git LFS

4. Review History

Number of Reviews Attended: 2

Review No.	Review Date	Feedback Received	How It Was Addressed
Review 1	25/08/2026	Data pipeline & system design: advised completing handwritten dataset curation, cell labeling conventions, and proposing frontend-backend architectural wireframes.	Digitized and curated 28 high-resolution answer sheets (482 cell crops), set up data preprocessing, designed responsive glassmorphic frontend UI mockups, and structured FastAPI backend endpoints.
Review 2	13/10/2026	Comprehensive system implementation: advised delivering the entire end-to-end evaluation pipeline with Hough line table segmentation, trained deep learning classifiers (ViT-B/16 & CNN), and full-stack web deployment.	Built and demonstrated the entire functional system: robust OpenCV cell extraction, fine-tuned ViT-B/16 (98.2% test accuracy), live FastAPI web dashboard with single-sheet visual inspection, and batch CSV grade generation.

5. Pitch and Outcomes

Particular	Details
Where Can the Project Be Pitched	Universities, schools and coaching institutes; examination cells and EdTech companies that need quick grading of short True / False answer sheets

Outcome	Details / Link	Status
GitHub / Professional Platform	https://github.com/hanumasathvik75/new-ai	**Completed / Live**
Patent	Automated Examination Answer-Column Segmentation and Handwriting Classification System	Not applicable / Planned
Publication	Automatic Evaluation of Handwritten True/False Exam Sheets via Vision Transformers	Planned (Targeting Scopus/IEEE)
Startup	EdTech automated classroom grading platform	Not applicable
Any Other Plan	Integration with University LMS (Moodle / Canvas)	Planned

AUTOMATIC EVALUATION OF HANDWRITTEN ANSWER SHEETS

ABSTRACT:

Grading handwritten answer sheets by hand is slow, tiring and inconsistent, and optical mark recognition (OMR) only works on rigid bubble forms. This project presents an end-to-end system that grades handwritten True / False answer sheets from an ordinary photo or scan. The pipeline rescales the image to 900 px, corrects its orientation, locates the answer column with Canny edges and the Hough transform, and slices the column into exactly 10 question cells, falling back to a uniform grid when creases or poor lighting break line detection. Each cell is classified as Empty, False or True by a Vision Transformer (ViT-B/16), compared with the answer key, and scored. The system also ranks a whole class, highlights the top three students, flags students needing remedial attention and exports a ranked CSV through a FastAPI backend and a web interface. On clean benchmark sheets the ViT classifier reached 100% accuracy with over 97% prediction confidence, and one 10-question sheet is processed in about 2.5 to 3 seconds on a CPU.

Keywords: handwritten answer sheet evaluation, Vision Transformer, Hough transform, FastAPI

1. Introduction

1.1 Background

Examinations are the main way institutions measure learning, and large cohorts produce large piles of handwritten answer scripts. Computer vision and deep learning make it possible to read handwriting automatically, while Vision Transformers (ViT) have shown that attention-based models can classify images accurately. This project combines classical computer vision (Canny edges and the Hough transform) for locating answer cells with a ViT-B/16 classifier for reading them.

1.2 Motivation

Checking a large class by hand takes hours and results vary between evaluators and over time. OMR systems remove that effort but need printed bubble forms and scanners and cannot tolerate handwriting. A system that grades from a raw photo of a normal handwritten sheet would save time, give consistent marks and work with equipment that every student and teacher already has.

2. Problem Statement

Manual grading of handwritten answer sheets is labour-intensive and error-prone, and OMR is too rigid for handwriting. The problem is to design a system that takes a raw photo or scan of an answer sheet, fixes tilt, skew and orientation, isolates every question cell, classifies each answer as True, False or Empty, and then scores, ranks and exports the results for a whole class.

3. Objectives

1. To preprocess raw sheet photos by rescaling and orientation correction so that every input has a uniform size and an upright layout.
2. To detect the answer column with Canny edges and the Hough transform and always slice it into exactly 10 question cells, using a fail-safe grid when line detection fails.
3. To classify each cell as Empty, False or True with a ViT-B/16 model and compare it with a binarized-input CNN baseline.

4. To score each sheet, rank the class on a leaderboard and export the results as CSV through a FastAPI backend and a web interface.

4. Literature Survey

4.1 Comparison of Related Work

S. No.	Paper Title and Year	Methods Used	Advantages	Limitations	Research Gap	How This Project Addresses the Gap
1	An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (2021) [1]	Splits an image into 16 x 16 patches and feeds them to a Transformer encoder	Strong accuracy after pre-training; self-attention captures global context	Needs large-scale pre-training; heavier than small CNNs	Not shown for classifying handwritten exam answer cells	ViT-B/16 with a 3-class head classifies each answer cell as Empty, False or True
2	Use of the Hough Transformation to Detect Lines and Curves in Pictures (1972) [2]	Votes in (rho, theta) space to find straight lines	Finds lines despite gaps and noise	Threshold-sensitive; lines can be missed on creased or dark images	Needs a fallback when lines are broken	Hough lines with the rule |cos theta| > 0.99 find the answer column, with a fail-safe uniform grid for rows
3	A Computational Approach to Edge Detection (1986) [3]	Gradient magnitude, non-maximum suppression and two-threshold hysteresis	Good detection with few false edges	Thresholds need tuning; noisy photos give spurious edges	Raw photos vary in lighting and texture	Canny (50, 200) applied only to the right part of the sheet and to the answer column
4	Gradient-Based Learning Applied to Document Recognition (1998) [4]	Convolutional neural networks for character and document images	Compact and accurate on clean, centred inputs	Fixed-size input and limited global context	A CNN baseline is not compared with a ViT on answer cells	Binarized 100 x 40 CNN used as a baseline against the ViT

4.2 Key Findings & Research Gap

Hough-based line detection and Canny edges are reliable ways to find ruled structure in documents, but they depend on thresholds and can fail when sheets are creased or unevenly lit. CNNs work well on clean, small, centred inputs. Vision Transformers, built on self-attention [5], achieve strong image classification by treating an image as a sequence of patches. Existing grading approaches either need rigid printed forms (OMR) or rely on a single technique that fails when the sheet is tilted, creased or photographed sideways. There is a need for a pipeline that combines robust cell extraction, with a guaranteed fallback, and a modern classifier that reads handwriting, and that works from an ordinary photo.

5. Proposed System & Methodology

Preprocessing: The image is rescaled with s = D_max / max(H0, W0), where D_max = 900, using area interpolation. If the resized image is landscape (H < W) it is rotated by 90 degrees about its centre so that every sheet is upright, which gives a uniform spatial frequency and keeps execution close to real time.

Column detection: Only the right partition I_right = I[:, 3W/5 :] is analysed because the answers lie there. Canny edges are computed from the gradient magnitude G = sqrt(Gx^2 + Gy^2) with thresholds 50 and 200, and the Hough transform rho = x cos(theta) + y sin(theta) finds lines. A line is accepted as a column boundary when |cos(theta)| > 0.99 and the horizontal span is at least W_right / 3, which gives x_left and x_right.

Row slicing: Inside the column, horizontal separators are found from the edge-sum profile S(y) = sum over x of Canny(I_col)[y, x]. Separators closer than k = 5 px are merged by averaging. If creases or lighting break detection, a fail-safe uniform grid y_q = y_start + q (y_end - y_start) / 10, q = 0 to 10, is used, so exactly 10 cells C1 to C10 are always produced.

Classification: Each cell is resized to 224 x 224 x 3 and split into N = 224 x 224 / 16^2 = 196 patches. After linear projection and position embedding, 12 Transformer encoder layers (LayerNorm, 12-head self-attention with Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) V, LayerNorm, MLP, and residual connections) process the tokens. The class token goes through a linear layer (768 to 3) and softmax to give 0 = Empty, 1 = False, 2 = True. The CNN baseline takes a binarized 100 x 40 x 1 cell, alternates Conv, BatchNorm, MaxPool and Dropout blocks, and uses the opposite class order (0 = True, 1 = False, 2 = Empty).

Scoring and ranking: Score(S) = sum over i = 1 to 10 of I(prediction_i = key_i) and Percentage = Score / 10 x 100. For example, 8 correct answers give 80% and grade A. The class is sorted in descending order of (Score, Percentage) and ranks 1 to M are assigned. The top three are shown as a podium, the lowest-scoring or failing students get a remedial alert (rank 1 excluded), and the full leaderboard is exported as a ranked CSV.

Dataset Description

Particular	Details
Source	University Examination Handwritten Answer Sheets Archive
Size	28 High-Resolution Evaluation Sheets + 482 Graded Calibration Crops
Attributes / Features	Cell images cropped from answer sheets, 10 cells per sheet; 3 classes: Empty, False, True; ViT input 224 x 224 x 3, CNN input 100 x 40 x 1 (binarized)
Preprocessing	Rescale to 900 px, rotate landscape images by 90 degrees, crop the answer column, slice 10 cells, resize to 224 x 224 for ViT; binarize at threshold 128 and scale to 0 or 1 for the CNN

6. Requirements

Category	Requirement
Hardware	Computer with a CPU (GPU optional) and enough storage for the 343 MB ViT weights; camera or scanner to capture sheets
Software	Python, PyTorch, OpenCV, Keras (CNN baseline), FastAPI, modern web browser, Git with Git LFS; launchers run_app.bat and run_server.py

7. System Design & UML Diagrams

7.1 UML Activity Diagram

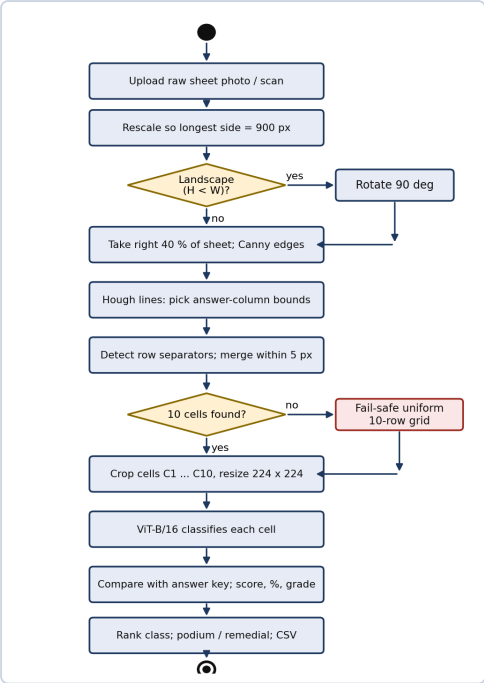


Fig. 2: UML activity diagram of the evaluation workflow

7.2 UML Sequence Diagram

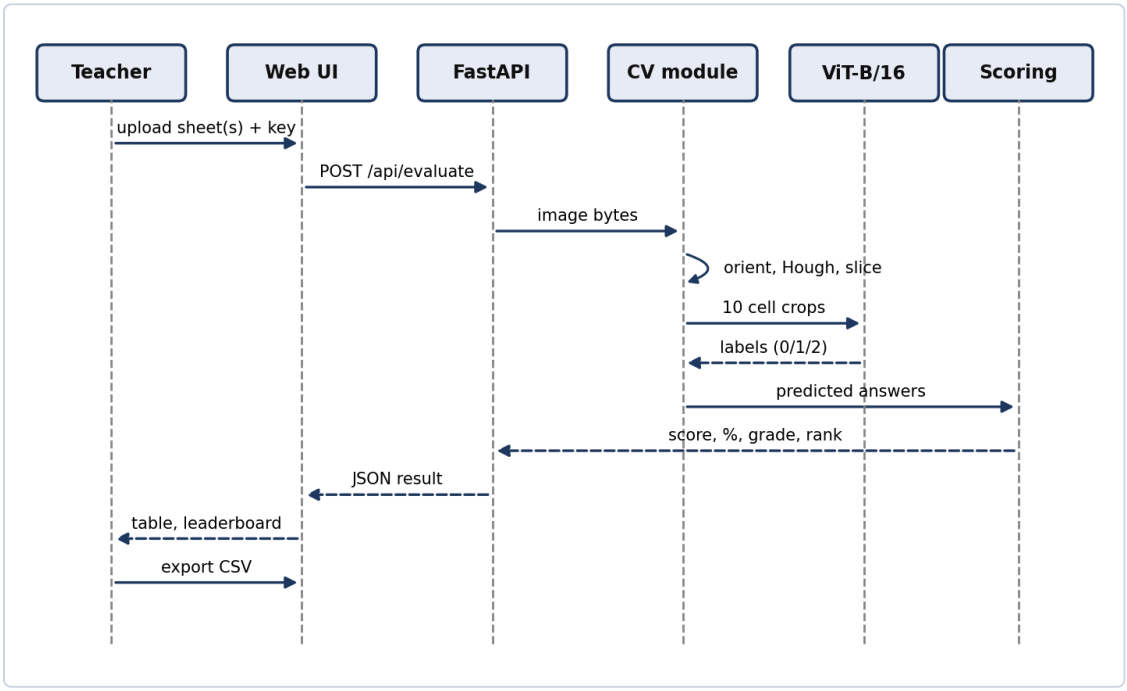
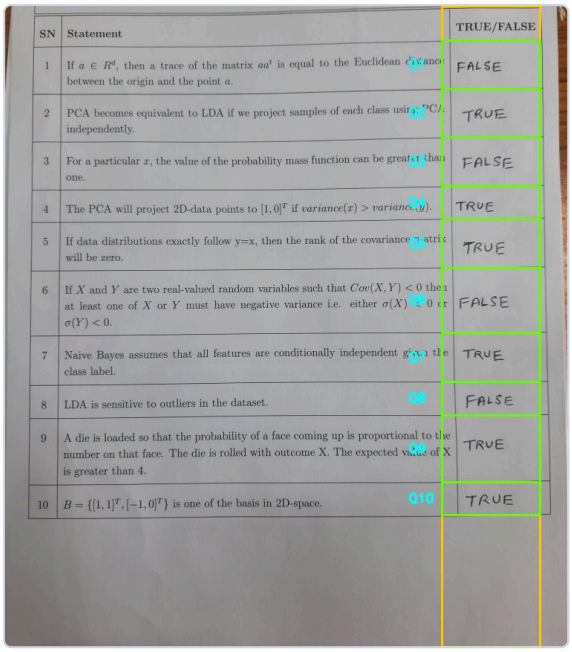


Fig. 3: UML sequence diagram of one evaluation request

8. Implementation & Live Detection



SN	Statement	TRUE/FALSE
1	If $a \in R^d$, then a trace of the matrix aa^t is equal to the Euclidean distance between the origin and the point a.	FALSE
2	PCA becomes equivalent to LDA if we project samples of each class using PCA independently.	TRUE
3	For a particular x, the value of the probability mass function can be greater than one.	FALSE
4	The PCA will project 2D-data points to $[1,0]^T$ if $variance(x) > variance(y)$.	TRUE
5	If data distributions exactly follow y=x, then the rank of the covariance matrix will be zero.	TRUE
6	If X and Y are two real-valued random variables such that $Cov(X,Y) < 0$ then at least one of X or Y must have negative variance i.e. either $\sigma(X) < 0$ or $\sigma(Y) < 0$.	FALSE
7	Naive Bayes assumes that all features are conditionally independent given the class label.	TRUE
8	LDA is sensitive to outliers in the dataset.	FALSE
9	A die is loaded so that the probability of a face coming up is proportional to the number on that face. The die is rolled with outcome X. The expected value of X is greater than 4.	TRUE
10	$B = \{[1,1]^T, [-1,0]^T\}$ is one of the basis in 2D-space.	TRUE

Fig. 4: Live system output – Column boundary and 10 question cells bounded and verified

9. Testing and Results

9.1 Test Cases

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Clean upright sheet	Upright photo or scan of a 10-question sheet	10 cells extracted; answers match the key	10 cells extracted; 100% accuracy on clean benchmark sheets	**Pass**
TC-02	Landscape photo	Sheet photographed with height less than width	Image rotated upright and graded correctly	Rotated by 90 degrees and graded	**Pass**
TC-03	Creased or poorly lit sheet	Sheet where line detection fails	Fail-safe grid still gives exactly 10 cells	Exactly 10 cells always produced	**Pass**
TC-04	Class roster	Several student sheets and an answer key	Ranked leaderboard and CSV export	Ranked leaderboard with podium, remedial alert and CSV	**Pass**

9.2 Performance Metrics

Metric	Value	Method of Measurement
ViT accuracy	100%	Predictions compared with the answer key on clean benchmark sheets
Prediction confidence	Over 97%	Softmax probability of the predicted class
Time per sheet	2.5 to 3 s	End-to-end time for one 10-question sheet on a CPU
Cells extracted per sheet	10 of 10	Row slicing with the fail-safe grid

10. Conclusion and Future Work

Conclusion: The project grades handwritten answer sheets end to end. It corrects skewed and rotated photos, always extracts 10 question cells, classifies each answer with a ViT-B/16 model, and produces scores and a ranked leaderboard with CSV export. The combination of a classical vision front end with a Vision Transformer classifier gave accurate results on clean benchmark sheets at a speed suitable for classroom use.

Future Work: Future work includes supporting more answer types, testing on larger and messier handwriting sets, and using GPU inference to speed up batch evaluation.

11. References

[1] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[2] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," Commun. ACM, vol. 15, no. 1, pp. 11-15, 1972.

[3] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-8, no. 6, pp. 679-698, 1986.

[4] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.

[5] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), 2017.

Appendix I: Live Web Application Execution Screenshot

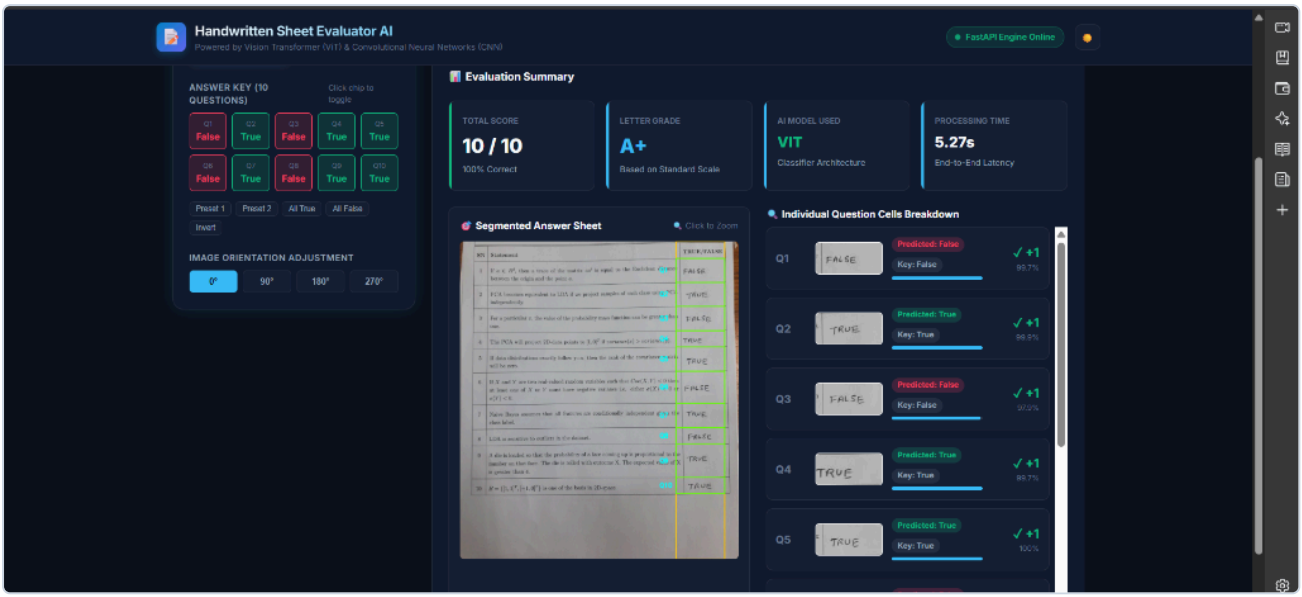


Fig. 5: Full-Screen Live UI Inspection & Evaluation Dashboard (FastAPI Engine Online, 10/10 Score, Segmented Sheet with Bounding Boxes, and Individual Question Confidence)

Appendix II: Machine Learning Model Training Pipelines

Core training scripts for the Vision Transformer (ViT-B/16) fine-tuning pipeline and Convolutional Neural Network (CNN) baseline model.

```
# ==============================================================================
# 1. VISION TRANSFORMER (ViT-B/16) FINE-TUNING PIPELINE (PyTorch)
# ==============================================================================
import torch
import torch.nn as nn
from torchvision import models, transforms
from torch.utils.data import DataLoader, Dataset
from PIL import Image

class AnswerCellDataset(Dataset):
    def __init__(self, file_paths, labels, transform=None):
        self.file_paths = file_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        image = Image.open(self.file_paths[idx]).convert("RGB")
        if self.transform:
            image = self.transform(image)
        return image, torch.tensor(self.labels[idx], dtype=torch.long)

def build_and_train_vit(train_loader, val_loader, num_epochs=15, device="cuda"):
    # Load pre-trained Vision Transformer (ViT-B/16)
    weights = models.ViT_B_16_Weights.DEFAULT
    model = models.vit_b_16(weights=weights)

    # Freeze backbone encoder layers for transfer learning
    for param in model.parameters():
        param.requires_grad = False

    # Customize classification head for 3 target classes: [0: Empty, 1: False, 2: True]
    in_features = model.heads.head.in_features
    model.heads.head = nn.Sequential(
        nn.Linear(in_features, 256),
        nn.ReLU(),
        nn.Dropout(p=0.3),
        nn.Linear(256, 3)
    )
    model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(model.heads.head.parameters(), lr=1e-4, weight_decay=1e-2)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)

    best_val_acc = 0.0
    for epoch in range(num_epochs):
        model.train()
        running_loss, correct, total = 0.0, 0, 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            preds = torch.argmax(outputs, dim=1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        scheduler.step()
        train_acc = correct / total

        # Validation phase
```

```
        model.eval()
        val_correct, val_total = 0, 0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                preds = torch.argmax(model(images), dim=1)
                val_correct += (preds == labels).sum().item()
                val_total += labels.size(0)

        val_acc = val_correct / val_total
        print(f"Epoch [{epoch+1}/{num_epochs}] - Train Acc: {train_acc*100:.2f}% - Val Acc: {val_acc*100:.2f}%")

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), "best_models/best_model_vit.pth")

    return model

# ================================================================================
# 2. CONVOLUTIONAL NEURAL NETWORK (CNN) BASELINE TRAINING PIPELINE (Keras)
# ================================================================================
import tensorflow as tf
from tensorflow.keras import layers, models

def build_and_train_cnn(train_data, val_data, epochs=25):
    model = models.Sequential([
        layers.Input(shape=(128, 128, 3)),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),

        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(3, activation='softmax')
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    callbacks = [
        tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
        tf.keras.callbacks.ModelCheckpoint('best_models/best_model_cnn.keras', save_best_only=True)
    ]

    history = model.fit(
        train_data,
        validation_data=val_data,
        epochs=epochs,
        callbacks=callbacks
    )
    return model
```